



INSTITUTO TECNOLOGICO

SUPERIOR DE LERDO

Desarrollo de Proyectos del semestre

Portafolio Desarrollo Móviles

Nombre:

Francisco Ivan Garcia Frayre

Carrera:

Informática

Docente:

Jesús Salas Marín

No° de control:

222311010

Fecha:

03/06/26

índice

Introducción.....	3
Desarrollo	4
Proyecto 1: Análisis de Datos con Dart.....	4
□ Capturas del proyecto:	5
Proyecto 2: Generador de Nombres Aleatorios	7
□ Capturas del proyecto	9
Proyecto 3: Mini Reproductor de Música	10
□ Capturas del proyecto	11
Reflexión Final.....	12
Conclusiones.....	13
Referencias	13

Introducción

Propósito del portafolio

El presente portafolio de evidencias tiene como propósito principal documentar, estructurar y presentar los proyectos integradores desarrollados durante el semestre en la asignatura de Desarrollo de Aplicaciones para Dispositivos Móviles. A través de este documento, se busca demostrar la evolución técnica y el dominio práctico de herramientas de vanguardia en la creación de software, transitando desde la lógica de consola con manipulación de datos crudos, hasta la implementación de interfaces gráficas reactivas.

Competencias desarrolladas Durante la construcción de estos proyectos, se lograron consolidar las siguientes competencias profesionales:

- **Diseño de Interfaces (UI/UX):** Construcción de interfaces responsivas y adaptables mediante el árbol de widgets de Flutter.
- **Gestión del Estado:** Implementación de arquitecturas de estado global (con Provider) para mantener la interfaz sincronizada con los datos de negocio.
- **Lógica de Programación y POO:** Uso avanzado de clases, constructores, manipulación de colecciones y lectura de archivos nativos.
- **Programación Reactiva y Asíncrona:** Manejo de flujos de datos (Streams) y funciones asíncronas para ejecutar procesos continuos en segundo plano.

 **Enlace al Repositorio (GitHub):**

<https://github.com/IvanGarcia24/PortafolioMoviles> IvanGarcia.git

Desarrollo

Proyecto 1: Análisis de Datos con Dart

- **Nombre del proyecto:** Análisis de Datos con Dart.
- **Objetivo del proyecto:** Construir un sistema mediante Dart capaz de leer información desde un archivo .json local, estructurarla utilizando Programación Orientada a Objetos y manipularla aplicando validaciones para exportar un reporte automatizado.
- **Problema planteado:** El análisis manual de datos crudos es propenso a errores por campos vacíos o mal formateados. Se requería automatizar la extracción y filtrado de grandes volúmenes de datos desde un archivo plano, evitando que el sistema colapsara por errores de entrada.
- **Tecnologías utilizadas:** Dart puro (Librerías dart:io y dart:convert).
- **Conceptos aplicados:** Programación Orientada a Objetos, Null Safety, manipulación de colecciones (clausuras y funciones de orden superior como .where()), lectura y escritura de archivos en disco.
- **Explicación del funcionamiento:** El programa inicia cargando un archivo datos.json desde el almacenamiento local. A través de un constructor factory, transforma el texto plano en una lista de objetos tipados en memoria. Posteriormente, despliega un menú interactivo en consola mediante un ciclo while. Dependiendo de la opción elegida por el usuario, el sistema itera la lista para buscar coincidencias (por nombre o rango salarial), calcula promedios y valores extremos, y finalmente serializa esos resultados en un nuevo archivo físico llamado resumen.json.

- Capturas del proyecto:

```
=== SISTEMA DE ANÁLISIS DE DATOS ===  
1. Ver todos los registros  
2. Buscar registro por nombre  
3. Filtrar por salario mínimo  
4. Generar reporte estadístico y exportar  
5. Salir  
Elige una opción: 1
```

```
--- LISTA DE EMPLEADOS ---  
Nombre: Carlos | Edad: 28 | Salario: $15000.50  
Nombre: Ana | Edad: 35 | Salario: $22000.00  
Nombre: Luis | Edad: 22 | Salario: $12000.00  
Nombre: Maria | Edad: 40 | Salario: $28000.75  
Nombre: Pedro | Edad: 30 | Salario: $18500.20
```

Ilustración 1: Carga de Datos y Menú

```
=== SISTEMA DE ANÁLISIS DE DATOS ===
1. Ver todos los registros
2. Buscar registro por nombre
3. Filtrar por salario mínimo
4. Generar reporte estadístico y exportar
5. Salir
Elige una opción: 2
Ingresa el nombre a buscar: luis

--- RESULTADOS DE BÚSQUEDA ---
Nombre: Luis | Edad: 22 | Salario: $12000.00
```

Ilustración 2: Búsqueda y Filtrado

```
=== SISTEMA DE ANÁLISIS DE DATOS ===
1. Ver todos los registros
2. Buscar registro por nombre
3. Filtrar por salario mínimo
4. Generar reporte estadístico y exportar
5. Salir
Elige una opción: 4

--- ESTADÍSTICAS ---
Total de registros: 5
Promedio de salario: $19100.29
Edad mínima: 22 años
Edad máxima: 40 años
¡Reporte exportado exitosamente como 'resumen.json' en tu carpeta!
```

Ilustración 3: Exportación de Estadísticas

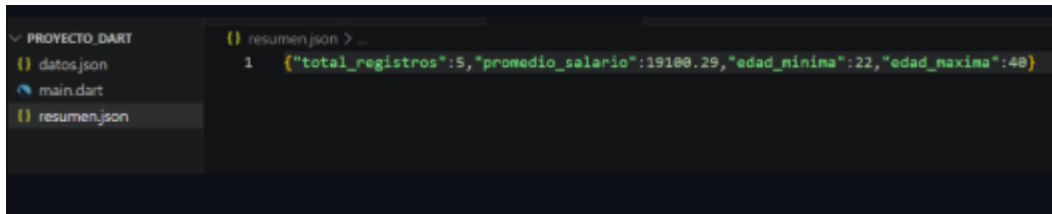


Ilustración 4: Archivo JSON Generado

- **Dificultades encontradas:** El mayor reto fue comprender el proceso de decodificación de datos: tomar el texto plano del archivo JSON, convertirlo a una estructura de tipo Mapa y usar iteraciones para instanciar la lista de objetos de forma segura sin romper el programa ante campos vacíos.
 - **Soluciones aplicadas:** Se implementó rigurosamente el operador ?? para asignar valores por defecto en caso de datos nulos y se protegió la captura de datos del usuario en consola usando `double.tryParse()`, garantizando que entradas alfabéticas no detuvieran la ejecución del programa.
-

Proyecto 2: Generador de Nombres Aleatorios

- **Nombre del proyecto:** Generador de Nombres Aleatorios.
- **Objetivo del proyecto:** Comprender los fundamentos del ecosistema Flutter mediante la creación de una aplicación interactiva responsiva que gestione un estado compartido entre diferentes pantallas.
- **Problema planteado:** En las aplicaciones modernas es necesario compartir información (como una lista de favoritos) entre diferentes pantallas sin perder los datos al navegar, evitando la fragmentación de la información en la interfaz.
- **Tecnologías utilizadas:** Flutter, Dart, paquetes `provider` y `english_words`.

- **Conceptos aplicados:** Gestión de Estado global (ChangeNotifier), Ciclo de vida de widgets (Stateless y Stateful), diseño responsivo (LayoutBuilder y NavigationRail), y componentización.
- **Explicación del funcionamiento:** La aplicación utiliza el paquete english_words para instanciar pares de palabras aleatorias. Al presionar el botón "Next", se actualiza la variable central y la interfaz reacciona al instante. Si el usuario presiona "Like", esa palabra se añade a un arreglo de favoritos administrado por Provider. Al utilizar el menú lateral (NavigationRail) para cambiar a la pestaña de favoritos, un ListView lee el arreglo global y renderiza todas las palabras guardadas previamente.

- **Capturas del proyecto:**



Ilustración 5: Generador de Nombres

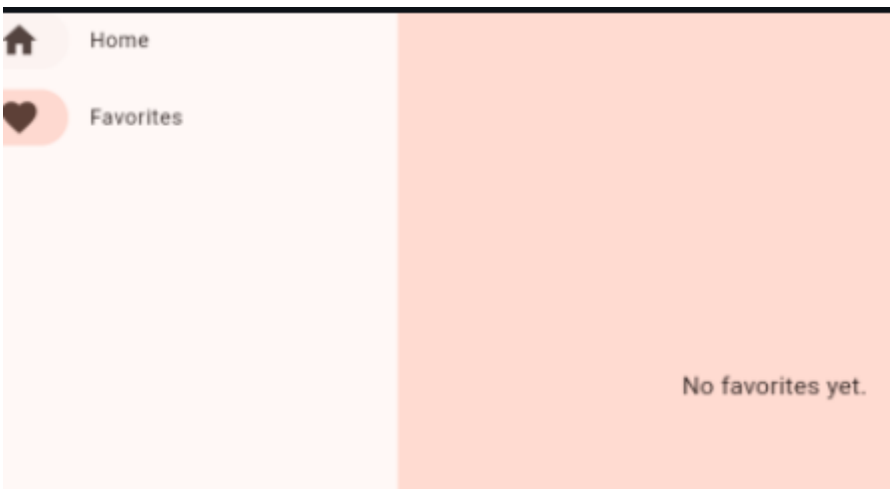


Ilustración 6: Lista de Favoritos

- **Dificultades encontradas:** Comprender la diferencia arquitectónica entre un StatelessWidget y un StatefulWidget, así como lograr el correcto anidamiento de widgets (Column, Row, Center) para que el diseño no se desbordara en distintas resoluciones de pantalla.

- **Soluciones aplicadas:** Se estudió la documentación sobre inyección de dependencias con Provider, extrayendo las funciones de actualización de la UI a una clase separada (MyAppState). Visualmente, se refactorizó el código agrupando la lógica de la tarjeta principal en un widget independiente llamado BigCard para mantener limpio el árbol principal.

Proyecto 3: Mini Reproductor de Música

- **Nombre del proyecto:** Mini Reproductor de Música.
- **Objetivo del proyecto:** Desarrollar una aplicación móvil que reproduzca archivos de audio locales, implementando programación reactiva para sincronizar los componentes visuales con los procesos ejecutados en segundo plano.
- **Problema planteado:** Mantener sincronizada una interfaz gráfica (como una barra de progreso multimedia) con el estado real de un motor de audio requiere actualizaciones constantes de la pantalla, lo que normalmente afectaría el rendimiento y la fluidez de la aplicación si no se gestiona correctamente.
- **Tecnologías utilizadas:** Flutter, Dart, paquetes just_audio y rxdart.
- **Conceptos aplicados:** Programación Reactiva (Streams), StreamBuilder, manejo de Assets multimedia locales y controladores asíncronos de estado.
- **Explicación del funcionamiento:** Al inicializar la pantalla, la aplicación carga un archivo .mp3 registrado en los assets. El estado de la reproducción, la posición actual y la duración total son emitidos como un flujo de datos continuo por el paquete de audio. Utilizando rxdart, estos tres flujos se combinan en un objeto único (PositionData). Finalmente, un StreamBuilder escucha este flujo combinado y reconstruye única y exclusivamente el widget de la barra de progreso varias veces por segundo, manteniendo la interfaz fluida y el audio sincronizado.

- **Capturas del proyecto:**



Ilustración 8: Pantalla Inicial



Ilustración 7: Reproducción en Curso

- **Dificultades encontradas:** El manejo avanzado de la asincronía. Fue complejo entender cómo combinar múltiples *Streams* (tiempo total, buffer y tiempo actual) para alimentar un solo componente gráfico sin generar "fugas de memoria" (memory leaks) por dejar procesos abiertos en segundo plano.
- **Soluciones aplicadas:** Se implementó una arquitectura reactiva pura mediante la librería rxdart para combinar flujos de información. Además, se hizo un manejo adecuado del ciclo de vida del widget sobrescribiendo el método `dispose()` para liberar correctamente los recursos del motor de audio cuando la pantalla se cierra.

Reflexión Final

- **¿Cuál proyecto fue más difícil?** El Mini Reproductor de Música. Salir de la programación secuencial tradicional para adentrarme en la programación reactiva (escuchar *Streams* y actualizar la pantalla en tiempo real) representó un salto importante en dificultad, especialmente al combinar múltiples fuentes de datos asíncronas para que la barra de progreso funcionara a la perfección.
- **¿Cuál fue más útil?** El Generador de Nombres Aleatorios. Esta práctica sentó las bases más importantes del desarrollo móvil por así decirlo: la gestión del estado. Entender cómo usar Provider para que un dato viva globalmente en la aplicación y pueda ser consultado desde cualquier pantalla cambió mi forma de estructurar el software.
- **¿Qué habilidad mejoró más?** La capacidad de construir lógicas orientadas a la prevención de errores y la abstracción visual. Mejoré significativamente en el uso de *Null Safety* y en la arquitectura de interfaces, entendiendo que el código de la vista (UI) debe estar estrictamente separado de la lógica de negocio.
- **¿Qué aprendiste sobre la asignatura?** Que el desarrollo de aplicaciones móviles va mucho más allá de diseñar pantallas atractivas. Requiere una planeación profunda sobre el ciclo de vida de la aplicación, un control riguroso de cómo viajan los datos entre componentes, y optimizar el rendimiento para no agotar los recursos del dispositivo móvil de los usuarios.

Conclusiones

El desarrollo de este portafolio representó una consolidación fundamental de los conocimientos adquiridos durante el octavo semestre. Evolucionar desde el manejo de datos en bruto mediante la terminal hasta orquestar aplicaciones interactivas con estados reactivos en Flutter, me proporcionó una visión integral de los retos reales del desarrollo de software actual. Las competencias adquiridas en el diseño de arquitecturas limpias y la resolución de problemas asíncronos me dejan una base técnica robusta y mucha seguridad para mis próximos pasos profesionales y proyectos de ingeniería.

Referencias

- Google LLC. (2026). *Documentación de Dart SDK*. Recuperado de <https://dart.dev>
- Google LLC. (2026). *Documentación de Flutter*. Recuperado de <https://flutter.dev>
- Rousselet, R. (2026). *Paquete provider* [Paquete Flutter]. [Pub.dev](https://pub.dev). Recuperado de <https://pub.dev/packages/provider>
- Heise, R. (2026). *Paquete just_audio* [Paquete Flutter]. [Pub.dev](https://pub.dev). Recuperado de https://pub.dev/packages/just_audio
- ReactiveX. (2026). *Paquete rxdart* [Paquete Dart]. [Pub.dev](https://pub.dev). Recuperado de <https://pub.dev/packages/rxdart>